Here are the solutions and explanations for your Power BI "Visual Calculations" homework puzzles. You can copy and paste this into a document to create your PDF.

Power BI Homework: Visual Calculations

Prerequisites: Download the `sales_with_geodata.csv` file.

Puzzle 1: Confusing Totals

Problem: The total of `Sales / Quantity` doesn't match the sum of individual rows.

Explanation: The problem arises from the difference between **row context** and **filter context**. When you create a calculated column like `Sales / Quantity`, Power BI calculates it for **each individual row** of the table. The "Total" at the bottom of the column then simply sums up these individual results.

However, when you use a measure, the total is calculated at a higher level (the total context of the visual). It's performing the calculation on the **aggregate totals** of `Sales` and `Quantity`, not on each row. The total is calculating `SUM(Sales) / SUM(Quantity)`.

Solution: You need to rewrite the calculation as a **measure** that iterates over each row of the table before summing the results. The `SUMX()` function is perfect for this.

DAX Measure:

```
Code snippet

Average Sale Price per Unit =

SUMX(

    'sales_with_geodata',

    'sales_with_geodata'[Sales] / 'sales_with_geodata'[Quantity]

)
```

Visual:

- **Table Visual**
- **Columns:** Product, Sales, Quantity, Average Sale Price per Unit (the new measure)
- **Result:** The total row of the Average Sale Price per Unit measure will now correctly match the sum of the individual rows.

Puzzle 2: Filtered vs. Unfiltered Totals

Task: Write two measures for a bar chart showing Total Sales per category and Total Sales ignoring the filter.

DAX Measures:

1. **Total Sales (Filtered):** This is a standard measure that automatically responds to the category on the axis.
2. Code snippet

```
Total Sales = SUM('sales_with_geodata'[Sales])
```

3.
4.
5. **Total Sales (All Categories):** This measure uses the ALL() function to remove the filter context from the Category column.
6. Code snippet

```
Total Sales (All Categories) =

CALCULATE(

    [Total Sales],

    ALL('sales_with_geodata'[Category])

)
```

7.
8.

Bonus: % of Total Column:

```
Code snippet

% of Total Sales =

DIVIDE(

    [Total Sales],

    [Total Sales (All Categories)]

)
```

Visual:

- **Bar Chart Visual**
- **Axis:** Category
- **Values:** Total Sales, Total Sales (All Categories), % of Total Sales
- **Result:** The Total Sales bar will change for each category, while the Total Sales (All Categories) bar will remain constant across all categories.

Puzzle 3: Changing Context with Slicers

Problem: The card visual for Total Sales changes when you select different countries.

Explanation: Measures operate within the **filter context** of the visual and the report page. When you select a country in the slicer, the slicer applies a filter to all visuals on the page (by default). The Total Sales measure then calculates the sum of sales **only for the rows that match the selected country**, causing the number on the card to change.

Follow-Up: Add a measure to ignore the slicer.

DAX Measure:

```
Code snippet

Total Sales (All Countries) =

CALCULATE(

    [Total Sales],

    ALL('sales_with_geodata'[Country])

)
```

Visual:

- **Card Visual 1:** Total Sales
- **Card Visual 2:** Total Sales (All Countries)
- **Slicer Visual:** Country
- **Result:** The first card will change as you select different countries in the slicer. The second card will always show the grand total of sales for all countries, regardless of the slicer selection.

Puzzle 4: Misleading Average

Problem: The `Average Sales = [Total Sales] / [Total Orders]` measure gives incorrect results in the visual.

Explanation: This calculation is incorrect because it is performed at the wrong level of granularity. The measure is calculating `SUM(Sales) / SUM(Orders)` for each **region**. This gives you the average sales per *order* in that region, not the average sales *per customer* or *per product*. The phrase "Average Sales per Order" is a bit ambiguous, but this measure is calculating total sales divided by total orders for the region as a whole. If you meant average sales per *line item*, you would need to iterate through each row as in Puzzle 1.

DAX Solution: To correctly calculate a row-level average (e.g., average sales per line item), you must iterate through the table using `AVERAGEX()`.

```
Code snippet
```

```
Average Sales per Unit =

AVERAGEX(

    'sales_with_geodata',

    'sales_with_geodata'[Sales] / 'sales_with_geodata'[Quantity]

)
```

Visual:

- **Table Visual**
- **Columns:** Region, Average Sales per Unit (new measure)
- **Result:** This measure now correctly averages the sales-per-unit for each individual transaction within the context of the region, providing an accurate average.

Puzzle 5: Highlight Top Product per Category

Task: Add a visual-level filter to show only the top-selling product per category.

DAX Measure:

```
Code snippet

Product Rank = RANKX(

    ALLEXCEPT(

        'sales_with_geodata',

        'sales_with_geodata'[Category]
```

```
    ),

    [Total Sales],

    ,

    DESC,

    Dense

)
```

Steps to Filter:

1. **Create the `Product Rank` measure.**
2. **Add the measure to the Matrix visual.** Place it in the "Values" field well.
3. **Apply a Visual-Level Filter:**
    - With the matrix selected, go to the "Filters" pane.
    - Find the `Product Rank` measure in the list of filters.
    - Change the filter type to **"Show items when the value is"** and set it to **"is 1"**.
    - Click "Apply filter."
4. **Result:** The matrix will now show only the single top-selling product for each category. You can then remove the `Product Rank` from the visual if you only want to use it for filtering.

Puzzle 6: Unexpected Blank Values

Problem: Some customers have blank values even though they made purchases.

Explanation: The measure `Sales in France = CALCULATE(SUM(Sales[Sales]), Sales[Country] = "France")` works correctly. The reason for blank values is that the measure is calculating the sum of sales **for each customer** within the `Sales` table. If a customer has made a purchase, but **not** in France, the `CALCULATE` function's filter (`Sales[Country] = "France"`) evaluates to `FALSE` for all of that customer's rows. The result is a blank value because there are no sales that meet the criteria.

How to Fix It: To show a zero instead of a blank, you can wrap the calculation in `IF()` and `ISBLANK()`.

DAX Measure:

```
Code snippet

Sales in France =

IF(

    ISBLANK(

        CALCULATE(

            SUM('sales_with_geodata'[Sales]),

            'sales_with_geodata'[Country] = "France"

        )

    ),

    0,

    CALCULATE(

        SUM('sales_with_geodata'[Sales]),

        'sales_with_geodata'[Country] = "France"

    )
```

)

Result: This measure will now display `0` for any customer who did not have sales in France, providing a clearer representation of the data.

Puzzle 7: Time Intelligence Confusion

Task: Add a line for previous month's sales to a line chart.

DAX Measure:

```
Code snippet

Previous Month Sales =

CALCULATE(

    [Total Sales],

    DATEADD(

        'sales_with_geodata'[OrderDate].[Date],

        -1,

        MONTH

    )

)
```

Challenge Handling Edge Cases:

- **First Month of Year:** DATEADD() correctly handles the beginning of the year, subtracting a month and moving to the previous year's December.
- **Missing Months:** If a month has no sales, DATEADD() will still correctly look for the previous month's data. If there are no sales in the previous month either, the measure will return a blank, which is the expected behavior.

Visual:

- **Line Chart Visual**
- **Axis:** OrderDate (by month)
- **Values:** Total Sales, Previous Month Sales
- **Result:** The line chart will now have two lines, one showing current month sales and the other showing sales from the previous month for comparison.

Puzzle 8: Row-Level Calculation

Problem: Why use SUMX() instead of just multiplying two columns?

Explanation: The formula Total Discount = SUMX(Sales, Sales[Quantity] * Sales[Discount per Unit]) is a classic example of **row context** within a measure.

- SUMX() is an **iterator function**. It tells Power BI to go through each **row** of the Sales table.
- For each row, it calculates the expression Sales[Quantity] * Sales[Discount per Unit].
- After it has performed this calculation for every single row, it then **sums** up all the individual results.

You **cannot** simply do SUM(Sales[Quantity]) * SUM(Sales[Discount per Unit]) because this would first sum all quantities and all discounts in the entire table (or the current filter context) and then multiply those two grand totals. This would produce an incorrect result, as it doesn't account for the unique discount applied to each quantity sold.

Conclusion: SUMX() correctly applies the calculation at the row level and then aggregates the result, which is the desired outcome for this type of calculation.

Puzzle 9: Rank with Ties

Task: Use RANKX() to handle ties correctly and allow descending/ascending logic.

DAX Measure:

```
Code snippet
```

```
City Sales Rank =

RANKX(

    ALL('sales_with_geodata'[City]),

    [Total Sales],

    ,

    DESC,

    DENSE

)
```

Explanation of Parameters:

- `ALL('sales_with_geodata'[City])`: This is the table over which to rank. `ALL()` removes any existing filters on the `City` column, so the ranking is performed across *all* cities, not just the ones visible in the current filter context.
- `[Total Sales]`: This is the expression to rank by.
- `,`: The third parameter is the value to rank. Leaving it blank means it uses the value from the current row in the iterator.
- `DESC`: Ranks from highest to lowest sales. Change to `ASC` for lowest to highest.
- `DENSE`: This is the key for handling ties. It assigns the same rank to tied values but doesn't skip the next number. For example, if two cities are tied for rank 2, they both get rank 2, and the next rank is 3 (instead of 4).

Visual:

- **Table Visual**
- **Columns:** `City`, `Total Sales`, `City Sales Rank`
- **Result:** The table will show a rank for each city, with tied ranks handled correctly.

Puzzle 10: Dynamic Titles and KPIs

Task: Show a dynamic card title that changes based on a slicer.

DAX Measure:

```
Code snippet

Selected Country Title =


"Sales for " & SELECTEDVALUE('sales_with_geodata'[Country], "All Countries")
```

Explanation of `SELECTEDVALUE()`:

- `SELECTEDVALUE()` returns the value when a single country is selected in the slicer.
- If multiple countries are selected, or if no country is selected, it returns the optional second argument, which we've set to `"All Countries"`.

Steps to Create the Dynamic Title:

1. **Create a new Card visual.**
2. **Add the `Selected Country Title` measure to the "Fields" well.**
3. **Adjust the Card Display:**
    - With the card selected, go to the "Format your visual" pane.
    - Expand `General` > `Title`.
    - Turn `Title` **off**. The measure itself will serve as the dynamic title.
4. **Add the Slicer:**
    - Create a new Slicer visual and add the `Country` field to it.

Result: When you select a country in the slicer, the card's title will dynamically update to reflect the selection, e.g., "Sales for United States," making the report more interactive and user-friendly.